

Développement Web Services

RMI (Remote Method Invocation)

RMI

- **RMI (Remote Method Invocation)** est une API Java qui permet d'appeler des méthodes d'objets situés sur des machines distantes comme si elles étaient locales.
 - C'est une forme de communication inter-processus qui facilite la création d'applications distribuées en Java.
 - Est un **middleware** Java qui permet de créer des applications orientées objets distribuées.
- **RMI fonctionne selon ce schéma :**
 - Un **serveur expose** des objets distants (remote objects)
 - Un **client obtient** une référence à ces objets
 - Le **client appelle des méthodes** sur ces objets comme s'ils étaient locaux

RMI

- **Composants clés :**

- **Interface Remote** : Définit les méthodes que le client peut appeler à distance
- **Implémentation serveur** : Classe qui implémente l'interface remote
- **Stub (proxy)** : Représentation côté client de l'objet distant
- **Skeleton** : Anciennement le composant côté serveur
- **RMI Registry** : Annuaire qui permet de localiser les objets distants

RMI

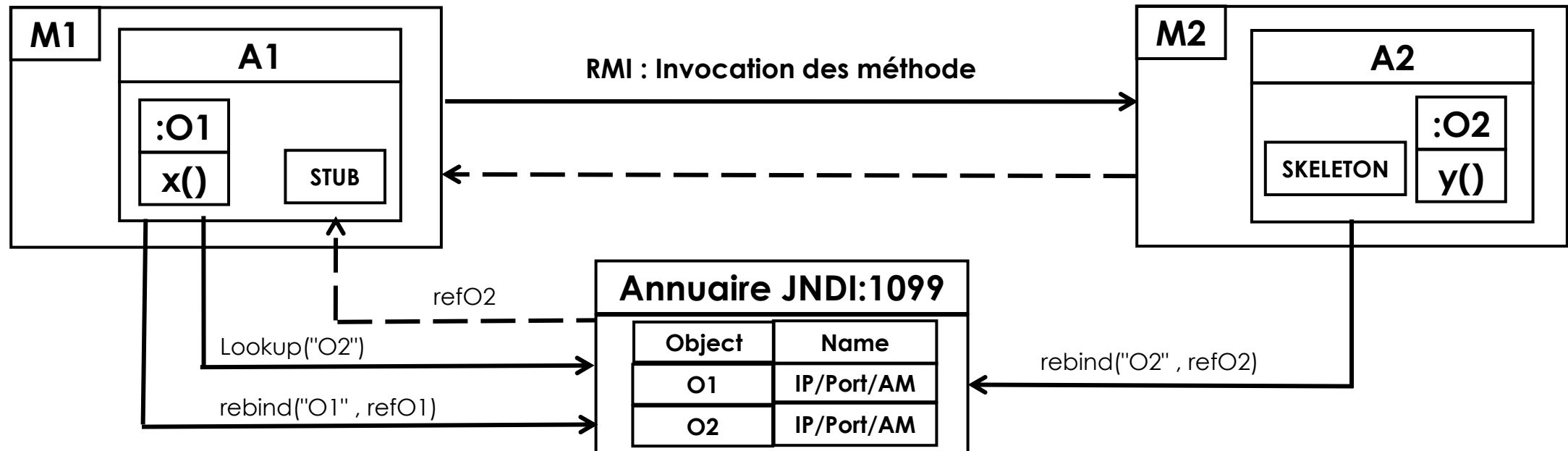
▪ Avec **RMI**, un objet Java **O1**, déployé dans une application **A1**, dans une machine **M1** peut faire appel aux méthodes d'un autre objet **O2**, déployé dans une application **A2**, dans une machine **M2** à travers des intermédiaires :

- **STUB** : Proxy côté Client
- **SKELETON** : Proxy côté Serveur

▪ Pour cela l'objet O1 a besoin de :

- L'interface de l'objet O2 (Interface Remote)
- La référence de l'objet O2 (IP + Port + Adresse mémoire)

Cette référence est publiée dans un annuaire JNDI (*Java Naming and Directory Interface*) en lui associant un nom fixe.



RMI

○ Avantages de RMI :

- **Transparent** : Le client appelle les méthodes comme si elles étaient locales
- **Sécurité** : Basé sur le système de sécurité de Java
- **Garbage collection distribué** : Via DGC (Distributed Garbage Collection)
- **Intégration Java** : Complètement intégré à l'environnement Java

○ Limitations :

- **Spécifique à Java** : Ne fonctionne qu'entre applications Java
- **Performances** : Moins performant que des solutions binaires
- **Complexité réseau** : Gestion des erreurs réseau peut être complexe

RMI

○ Règles principales à suivre pour implémenter RMI :

- Le développement **côté serveur** se compose de :
 - La définition d'une interface qui contient les méthodes qui peuvent être appelées à distance
 - L'écriture d'une classe qui implémente cette interface
 - L'écriture d'une classe quiinstanciera l'objet et l'enregistrera en lui affectant un nom dans le registre de noms RMI (RMI Registry)
- Le développement **côté client** se compose de :
 - L'obtention d'une référence sur l'objet distant à partir de son nom
 - L'appel à la méthode à partir de cette référence

RMI

Contraintes sur interface de méthodes :

- Spécialise **java.rmi.Remote** : précise qu'il s'agit d'une interface de service appelables à distance
 - Chaque méthode doit préciser dans sa signature qu'elle peut lever l'exception **RemoteException**
 - L'opération peut également lever des exceptions spécifiques à l'application
 - **RemoteException** est la classe mère d'une hiérarchie d'une vingtaine d'exceptions précisant un problème lors de l'appel de l'opération, comme, par exemple:
 - **NoSuchObjectException** : ne peut récupérer la ref sur l'objet distant
 - **StubNotFoundException** : un stub correct n'a pas été trouvé
 - **UnknownHostException** : la machine distante n'a pas été trouvée
 - **AlreadyBoundException** : le nom associé à l'objet est déjà utilisé
 - **ConnectException** : connexion refusée par la partie serveur
- Pour plus d'informations sur l'exception : opération `getCause()` de `RemoteException`

RMI

Servant : exportation

- La classe implémentant l'interface doit utiliser la classe **UnicastRemoteObject**
- Cette classe sert à la communication via TCP
- L'objet de la classe implémentant l'interface doit « s'exporter » pour accepter des connexions de clients
 - Création et utilisation des ressources et éléments nécessaires à la communication via des sockets TCP
- **Unicast** : l'objet de la classe implémentant l'interface n'existe qu'en un seul exemplaire sur une seule machine
- L'objet meurt avec la fin de l'exécution du serveur qui le lance
- Deux modes
 - La classe étend directement la classe **UnicastRemoteObject**
 - Le constructeur par défaut appelle **super()**;
 - Pas de spécialisation et le constructeur doit exécuter l'instruction suivante :
UnicastRemoteObject.exportObject(this);
- Le constructeur par défaut doit préciser dans sa signature qu'il peut lever **RemoteException**

RMI

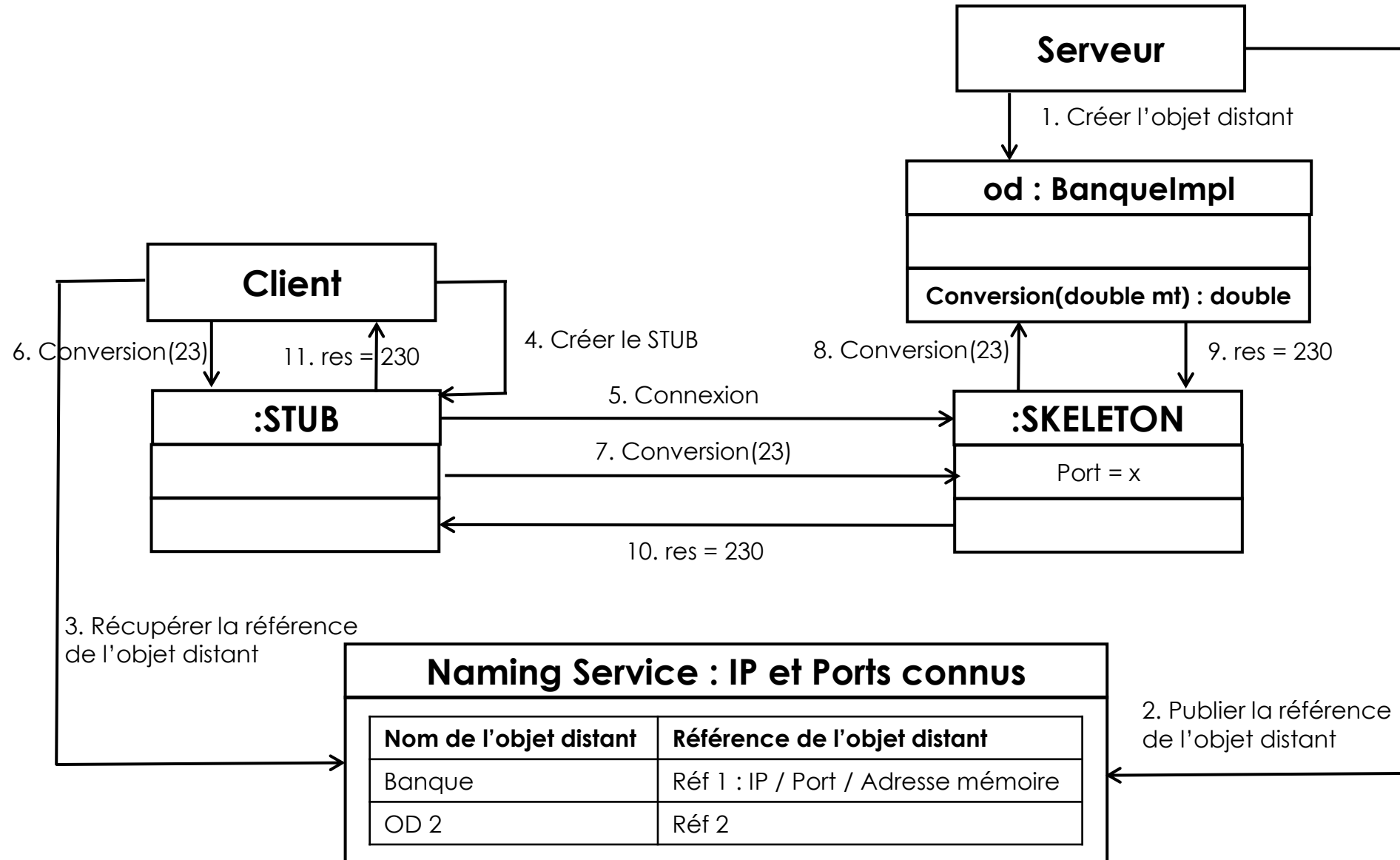
Registry

- Une partie client doit pouvoir récupérer une référence sur l'objet distant implémentant l'interface
 - Utilise les services du « **registry** » (registre, annuaire ...)
 - Le registry est lancé à part à côté des applications Java
 - Un objet accède au registry via la classe **Naming**
- Identification d'un objet distant
 - Via une URL de la forme **rmi://hote:port/nomObj**
 - hote : nom de la machine distante (sur laquelle tourne un registry)
 - port : port sur lequel écoute le registry
 - nomObj : nom donné à un objet offrant des opérations
- Si pas précision du port : port par défaut utilisé par le registry
- Si pas précision de l'hôte : registry local par défaut (localhost)

Objets distribués

- Un objet « distribué » est accessible à distance
 - D'un point de **vue type**, il implémente indirectement l'interface Remote
 - D'un point de **vue technique**, il s'est rendu visible sur le réseau en s'exportant via les services de UnicastRemoteObject
- Concernant les paramètres d'appel d'une méthode ou sa valeur de retour
 - **Types primitifs** : passage par valeur comme en local
 - **Classe implémentant Remote** : passage par référence
 - Référence sur un objet distant = référence locale sur son stub
 - **Classe n'implémentant pas Remote** : passage par valeur (copie)
 - D'où l'obligation d'implémenter Serializable pour transiter par des sockets
- Le registry RMI n'est qu'un outil facilitant si besoin la récupération d'une référence sur un objet distant
 - Aucune obligation d'enregistrer tous les objets distribués sur un registry

Etapes de réalisation d'un projet RMI



TP 2 - RMI

Travail à réaliser (1/3)

- 1) Créer un projet Java « **TP2-RMI** »
- 2) Créer une interface « **IBanqueRemote** » qui hérite de la classe « **Remote** » (de **Java.rmi**) (la mettre dans un package « **tp.rmi** »)

- Ajouter dans cette interface les signatures de méthodes suivantes :

```
public double conversion(double montant) throws RemoteException ;  
public Compte consulterCompte(int code) throws RemoteException ;  
public List<Compte> listComptes() throws RemoteException ;
```

- 3) Créer la classe « **Compte** » qui implémente l'interface « **Serializable** » (la mettre dans un package « **tp.metier** »)

- Ajouter à cette classe les attributs suivants :

```
int code / double solde / Date dateCreation (+ Constructeurs / Getters & Setters)
```

TP 2 - RMI

Travail à réaliser (2/3)

- 4) Créer la classe « **BanqueRMIService** » (dans le package « **tp.rmi** ») qui hérite de la classe « **UnicastRemoteObject** » et qui implémente l'interface « **IBanqueRemote** »
 - Implémenter les méthodes de cette classe + ajouter un constructeur par défaut
- 5) Créer la classe « **ServeurRMI** » (avec la méthode « `main(...)` ») dans le package par défaut
 - Lancer l'annuaire JNDI avec : `LocateRegistry.createRegistry(1099) ;`
 - Instancier un objet de la classe « BanqueRMIService » (puis l'afficher)
 - Ajouter le dans l'annuaire avec : `Naming.rebind("nom" , objet) ;`
- 6) Exécuter le projet (pour lancer le serveur et voir la référence de l'objet distant)
- 7) Créer un nouveau projet « **RMIClient** » (lui ajouter la classe « Compte » et l'interface « IBanqueRemote »)
- 8) Créer la classe « **ClientRMI** » (avec la méthode « `main(...)` »)

TP 2 - RMI

Travail à réaliser (3/3)

- Se connecter à l'annuaire avec : `Naming.lookup("adresse")` ;
- Appeler les 3 méthodes distantes
- Exécuter le Serveur et le Client

9) Enlever la classe « Compte » et l'interface « IBanqueRemote » du projet client

- Générer un fichier .jar du serveur
- L'ajouter au projet client
- Exécuter le tout